

DÉVELOPPEMENT WEB JS & PHP

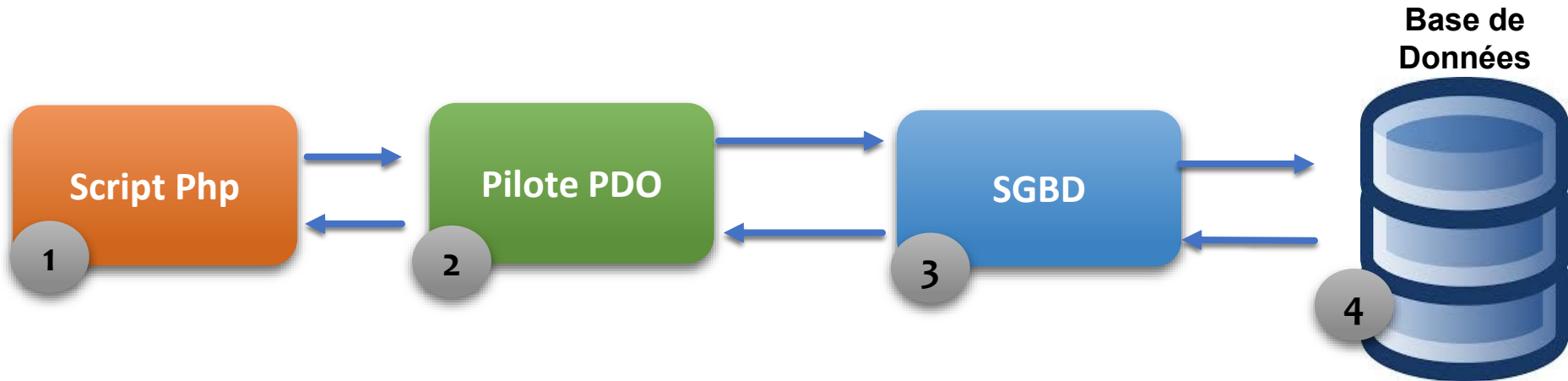
PHP- Chapitre 5

Accès aux bases de données en PHP **PDO**



Connexion aux bases de données

- Le script PHP (1) ne dialogue pas directement avec le SGBD (Système de Gestion de Bases de Données) (3).
- On utilise un intermédiaire appelé pilote de SGBD ou encore driver de SGBD (un driver pour chaque SGBD).
- PHP fournit une interface standard pour ces pilotes, l'interface PDO (PHP Data Objects).



PDO

Qu'est-ce que PDO ?

PDO = PHP Data Objects

Extension PHP **officielle** d'accès aux bases de données

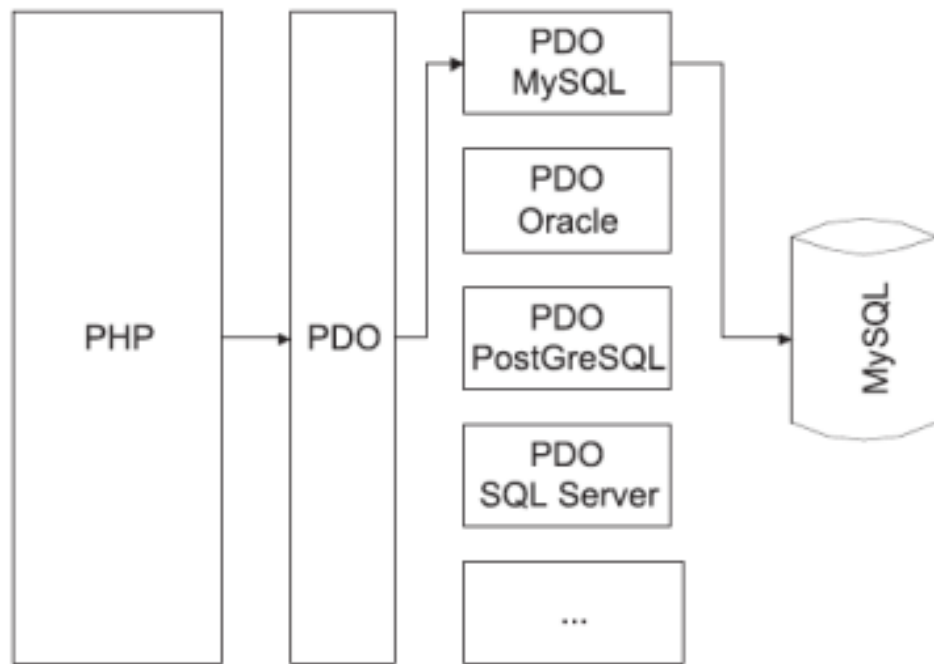
Interface unifiée pour plusieurs SGBD :
MySQL, PostgreSQL, SQLite, Oracle...

Introduit en **PHP 5.1** (2005)
Standard depuis PHP 7+

Pourquoi PDO ?

- ✓ **Sécurité**
Protection contre SQL injection via requêtes préparées
- ✓ **Portabilité**
Changer de SGBD sans réécrire le code
- ✓ **Performance**
Requêtes préparées exécutées plusieurs fois
- ✓ **Gestion d'erreurs**
Exceptions PDO claires et contrôlables

Architecture PHP ↔ Base de Données



✓ PDO (PHP Data Objects) — API unifiée, indépendante du SGBD. Recommandée.

⚠ MySQLi — spécifique à MySQL, orientée objet ou procédurale. Moins portable.

PDO : les classes à utiliser

Classe	Rôle
PDO	gère l'accès aux SGBD ainsi que les fonctionnalités de base : Connexions, lancement des requêtes...
PDOStatement	La classe gère une liste de résultats d'une requête Sql
PDOException	Une classe d'exception personnalisée interne pour PDO

Traitement des exceptions

- Les exceptions permettent de gérer les erreurs d'exécution.
- Une exception définit un espace dans lequel les erreurs sont détectées.
- Les exceptions permettent de mieux organiser la gestion des erreurs et des événements.

```
try {  
    // traitement  
}  
catch (PDOException $e) {  
    // traitement d'une erreur base de données  
}  
catch (Exception $e) {  
    // Traitement d'une erreur inconnue  
}
```

Connexion à la base de données

DSN (Data Source Name) : **"driver:host=...;dbname=...;charset=..."**

```
$host = 'localhost';  
$dbname = 'ma_base';  
$user = 'root'; $password = '';  
  
try {  
    $dsn = "mysql:host=$host;dbname=$dbname;charset=utf8mb4";  
    $pdo = new PDO($dsn, $user, $password, [  
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
    ]);  
    echo "Connexion réussie !\n";  
} catch (PDOException $e) {  
    die("Erreur : " . $e->getMessage());  
}
```

Exécution de requêtes simples

`$pdo->query()`

SELECT — retourne un PDOStatement

```
$stmt = $pdo->query(
    "SELECT * FROM users"
);
$users = $stmt->fetchAll();
```

`$pdo->exec()`

INSERT / UPDATE / DELETE — retourne le nombre de lignes affectées

```
$nb = $pdo->exec(
    "DELETE FROM logs
    WHERE date < '2024-01-01'"
);
echo "$nb lignes supprimées";
```


Requêtes préparées — prepare()



Sécurité

Évite les injections SQL



Performance

Compilée une fois, exécutée N fois



Lisibilité

Code clair, paramètres séparés

```
// Paramètres nommés (:param)
$stmt = $pdo->prepare("INSERT INTO users (nom, email, age)
                      VALUES (:nom, :email, :age)");

// Liaison des paramètres
$stmt->bindParam(':nom',    $nom,    PDO::PARAM_STR);
$stmt->bindParam(':email',  $email,  PDO::PARAM_STR);
$stmt->bindParam(':age',    $age,    PDO::PARAM_INT);

$stmt->execute(); // Exécution sécurisée

// OU : passage direct du tableau
$stmt->execute([':nom'=>$nom, ':email'=>$email, ':age'=>$age]);
```

Requêtes préparées — prepare()



Sécurité

Évite les injections SQL



Performance

Compilée une fois, exécutée N fois



Lisibilité

Code clair, paramètres séparés

```
// Les marqueurs positionnels ?
$stmt = $pdo->prepare("INSERT INTO users (nom, email, age)
                      VALUES ( ?, ?, ?)");

// Liaison des paramètres
$stmt->bindParam( 1 , $nom, PDO::PARAM_STR);
$stmt->bindParam( 2 , $email, PDO::PARAM_STR);
$stmt->bindParam( 3 , $age, PDO::PARAM_INT);

$stmt->execute(); // Exécution sécurisée

// OU : passage direct du tableau
$stmt->execute([$nom, $email, $age]);
```

Sécurité — SQL Injection

✗ Code DANGEREUX (sans PDO)

```
$id = $_GET['id']; // ex: "1 OR 1=1"
$sql = "SELECT * FROM users WHERE id = $id";
$result = mysql_query($sql);
// DANGER ! L'attaquant peut tout lire/modifier
```

✓ Code SÉCURISÉ (avec PDO)

```
$id = $_GET['id'];
$stmt = $pdo->prepare(
    "SELECT * FROM users WHERE id = :id"
);
$stmt->execute([':id' => $id]);
```

⚠ **Exemple d'attaque :** GET ?id=1 OR 1=1 → retourne TOUS les utilisateurs !

Comment PDO protège :

- Les paramètres sont envoyés **séparément de la requête SQL** au serveur de base de données
- Le pilote échappe automatiquement tous les caractères spéciaux (apostrophes, guillemets...)
- Il est **impossible** d'injecter du SQL malveillant dans un paramètre préparé

Récupération des données

Mode de fetch	Description	Résultat
<code>PDO::FETCH_ASSOC</code>	Tableau associatif (noms colonnes)	<code>['nom' => 'Alice', 'email' => '...']</code>
<code>PDO::FETCH_NUM</code>	Tableau numérique (indices)	<code>['Alice', 'alice@mail.com', 25]</code>
<code>PDO::FETCH_OBJ</code>	Objet stdClass	<code>\$row->nom, \$row->email</code>
<code>PDO::FETCH_CLASS</code>	Instance d'une classe	<code>new User(nom: 'Alice', ...)</code>

Récupération des données

```
// — 1. Lire tous les enregistrements —  
$stmt = $pdo->query('SELECT * FROM etudiants ORDER BY nom');  
$etudiants = $stmt->fetchAll(); // tableau de tableaux  
  
foreach ($etudiants as $e) {  
    echo $e['nom'] . ' - ' . $e['note'] . '/20';  
}  
  
// — 2. Filtrer avec paramètres —  
$stmt = $pdo->prepare(  
    'SELECT * FROM etudiants WHERE filiere = :f AND note >= :min'  
);  
$stmt->execute([':f' => 'INFO', ':min' => 10]);  
$admis = $stmt->fetchAll();  
echo count($admis) . ' admis en INFO';  
  
// — 3. Un seul enregistrement —  
$stmt = $pdo->prepare('SELECT * FROM etudiants WHERE id = ?');  
$stmt->execute([$id]);  
$etudiant = $stmt->fetch(); // false si introuvable  
if (!$etudiant) die('Étudiant introuvable.');
```

Méthodes de lecture

fetchAll()

Tous les résultats dans un array

fetch()

Un seul enregistrement (false si vide)

fetchColumn(n)

Valeur de la n-ième colonne

fetchObject(\$class)

Résultat comme objet d'une classe

rowCount()

Nb de lignes affectées (UPDATE/DELETE)

columnCount()

Nb de colonnes dans le résultat

Récupération des données

```
// — 4. Compter les résultats —————  
$stmt = $pdo->query('SELECT COUNT(*) FROM etudiants');  
$total = $stmt->fetchColumn(); // première colonne  
  
// — 5. Fetch en objet —————  
$stmt = $pdo->query('SELECT * FROM etudiants LIMIT 5');  
while ($row = $stmt->fetch(PDO::FETCH_OBJ)) {  
    echo $row->nom . ' : ' . $row->email;  
}
```

Méthodes de lecture

fetchAll ()

Tous les résultats dans un array

fetch ()

Un seul enregistrement (false si vide)

fetchColumn (n)

Valeur de la n-ième colonne

fetchObject (\$class)

Résultat comme objet d'une classe

rowCount ()

Nb de lignes affectées (UPDATE/DELETE)

columnCount ()

Nb de colonnes dans le résultat

Récupération des données: PDO::FETCH_CLASS

1. Définir la classe métier

```
<?php
class Etudiant {
    // Propriétés = noms des colonnes SQL
    public int    $id;
    public string $nom;
    public string $prenom;
    public string $filiere;
    public float  $note_moyenne;

    public function getMention(): string {
        return match(true) {
            $this->note_moyenne >= 16 => 'Très
bien',
            $this->note_moyenne >= 14 => 'Bien',
            default                    =>
'Passable',
        };
    }
}
```

2. Récupérer avec FETCH_CLASS

```
$stmt = $pdo->prepare(
    "SELECT * FROM etudiants WHERE filiere = :f"
);
$stmt->execute([':f' => 'Informatique']);

// Retourne un tableau d'objets Etudiant
$etudiants = $stmt->fetchAll(
    PDO::FETCH_CLASS,
    Etudiant::class
);

// Utilisation OO directe :
foreach ($etudiants as $e) {
    echo $e->nom . " : " . $e->getMention();
}
```

PDO mappe automatiquement les noms de colonnes SQL vers les propriétés de la classe — **les noms doivent correspondre exactement.**

Transactions

Une **transaction** = ensemble d'opérations SQL qui doivent toutes réussir ou toutes échouer (principe **ACID**).

`beginTransaction()`

Démarre la transaction

`execute() × N`

Exécute les requêtes

`commit()`
ou `rollback()`

Valide ou annule

```
try {  
    $pdo->beginTransaction();  
  
    // Virement bancaire : débit + crédit doivent réussir ensemble  
    $pdo->exec("UPDATE comptes SET solde = solde - 500 WHERE id = 1");  
    $pdo->exec("UPDATE comptes SET solde = solde + 500 WHERE id = 2");  
  
    $pdo->commit(); // Tout s'est bien passé → on valide  
} catch (PDOException $e) {  
    $pdo->rollBack(); // Erreur → on annule tout  
    throw $e;  
}
```


Gestion des erreurs — PDOException

Silencieux (défaut)

`PDO::ERRMODE_SILENT`

Aucune erreur affichée
Vérifier manuellement

Avertissement

`PDO::ERRMODE_WARNING`

Affiche un PHP Warning
Continue l'exécution

Exception (recommandé)

`PDO::ERRMODE_EXCEPTION`

Lance une PDOException
Arret propre du script

```
// Configuration recommandée lors de la connexion
$pdo = new PDO($dsn, $user, $pass, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
]);

// Exploitation de PDOException
catch (
    PDOException $e) {
    error_log($e->getMessage());
    echo "Une erreur est survenue. Code : " . $e->getCode();
}
```

Exemple complet — CRUD avec PDO

C - CREATE

```
$stmt = $pdo->prepare(
    "INSERT INTO produits(nom,prix)
    VALUES(:nom,:prix)"
);
$stmt->execute([':nom'=>$n,':prix'=>$p]);
```

R - READ

```
$stmt = $pdo->prepare(
    "SELECT * FROM produits
    WHERE categorie = :cat"
);
$stmt->execute([':cat'=>$cat]);
$rows = $stmt->fetchAll();
```

U - UPDATE

```
$stmt = $pdo->prepare(
    "UPDATE produits SET prix=:prix
    WHERE id = :id"
);
$stmt->execute([':prix'=>$p,':id'=>$id]);
```

D - DELETE

```
$stmt = $pdo->prepare(
    "DELETE FROM produits
    WHERE id = :id"
);
$stmt->execute([':id' => $id]);
```

Bonnes Pratiques & Récapitulatif

Toujours utiliser des requêtes préparées

Ne jamais concaténer des variables utilisateur dans le SQL

Configurer ERRMODE_EXCEPTION

Attraper les PDOException dans des blocs try/catch

Fermer la connexion

Mettre \$pdo = null après utilisation

Spécifier le charset

charset=utf8mb4 dans le DSN pour l'Unicode complet

Utiliser les transactions

Pour toute opération multi-requêtes liées

Centraliser la connexion

Créer une classe ou un fichier db.php réutilisable

Annexe

La table de référence — étudiants

```
-- Création de la table de référence
```

```
CREATE TABLE etudiants (  
    id            INT PRIMARY KEY AUTO_INCREMENT,  
    nom           VARCHAR(50) NOT NULL,  
    prenom        VARCHAR(50) NOT NULL,  
    filiere       VARCHAR(30) DEFAULT 'Informatique',  
    note_moyenne  FLOAT          DEFAULT 0.0  
);
```

```
-- Données initiales
```

```
INSERT INTO etudiants (nom, prenom, filiere, note_moyenne) VALUES  
    ('Amrani', 'Karim', 'Informatique', 15.5),  
    ('Belhaj', 'Sana', 'Réseaux', 12.0),  
    ('Cherif', 'Youssef', 'Informatique', 17.2),  
    ('Dridi', 'Ines', 'Logiciel', 9.8);
```

Données exemples

id	nom	filière	note
1	Amrani	Informatique	15.5
2	Belhaj	Réseaux	12.0
3	Cherif	Informatique	17.2
4	Dridi	Logiciel	9.8

Rappel SQL — SELECT

```
SELECT colonne1, colonne2 FROM table [WHERE condition] [ORDER BY col] [LIMIT n]
```

READ

Tous les étudiants

```
SELECT * FROM etudiants;
```

→ Retourne toutes les colonnes et toutes les lignes

Filtrer par filière

```
SELECT nom, prenom, note_moyenne  
FROM etudiants  
WHERE filiere = 'Informatique'  
ORDER BY note_moyenne DESC;
```

→ Retourne Cherif (17.2) puis Amrani (15.5)

Moyenne par filière

```
SELECT filiere,  
       AVG(note_moyenne) AS  
moyenne,  
       COUNT(*)           AS  
nb_etudiants  
FROM etudiants  
GROUP BY filiere;
```

→ Résumé statistique par filière

Rappel SQL — INSERT / UPDATE / DELETE

INSERT

```
-- Ajouter un étudiant
INSERT INTO etudiants
    (nom, prenom, filiere,
    note_moyenne)
VALUES
    ('Mansouri', 'Leila',
    'Informatique', 14.5);
```

⚠ Retourne l'id auto-généré via LAST_INSERT_ID()

UPDATE

```
-- Modifier la note d'un étudiant
UPDATE etudiants
SET     note_moyenne = 16.0,
        filiere       = 'Réseaux'
WHERE   id = 2;
```

⚠ Sans WHERE → tous les enregistrements modifiés !

DELETE

```
-- Supprimer un étudiant
DELETE FROM etudiants
WHERE id = 4;

-- Supprimer par critère
DELETE FROM etudiants
WHERE note_moyenne < 10.0;
```

⚠ Sans WHERE → toute la table vidée !